

INF122 Final Design Document

Emily Tang, Claire Phan, Bryant Dang, Jada Thoeung-Ferguson
17 March 2026

GitHub

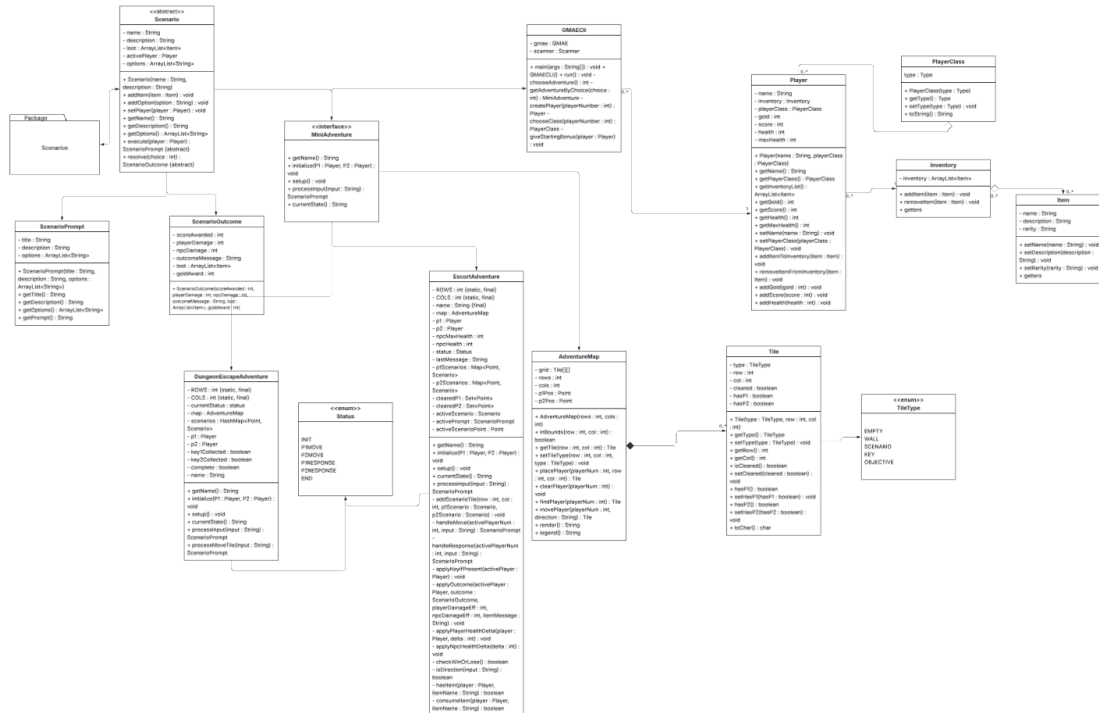
[GitHub Link](#)

*the Jar file is in our Github

Note: Professor and TAs were added as collaborators to our GitHub.

Please reach out to Emily Tang (tange8@uci.edu) if you need access to our GitHub! Thank you!

LUCIDCHART LINK



2. GuildQuest Overview

A comprehensive overview of our GuildQuest System. GuildQuest is a two-player command-line game system built around a set of modular mini-adventures. Players choose a mini-adventure and are dropped into a tile-based map where they navigate using cardinal directions, encounter scenario events, collect items, and work toward a shared objective.

2.1 General CLI Navigation

Welcome Menu

Users are welcomed with our welcoming message and asked to input player names. Users are also required to determine player classes for each player.

Player Profiles

Users should keep in mind the various attributes of the player itself when engaging with a game. Scenario outcomes, gold thresholds, and available options are all directly influenced by the player's current state. For example, a mini game may affect a player's health, gold, score, inventory, and class-specific capabilities.

Player Attribute	Description
Health	Starts at 100. Damaged by scenario choices; restored through healing scenarios or items. Falling to 0 ends the run.
Gold	Starts at 0. Gained or lost through scenario outcomes. Some options are gated behind a minimum gold balance, so managing gold is part of survival.
Inventory	Holds items collected during the run (e.g. Bandage, Medkit). Items acquired from scenario loot drops and can be used in future encounters.
Class	Warrior, Mage, or Rogue. Determines which options perform best in each scenario. Choosing actions that match your class yields better rewards and less damage.

2.3 GuildQuest System

Users will be given a game menu with the options being: "Dungeon Escape Adventure" or "Escort Adventure". Each game uses our AdventureMap foundation, a two-dimensional grid-based map for our

mini adventures. The map maintains various states via Tiles, which allow objects to be represented and tracked.

```

Legend: [1]=P1 [2]=P2 [B]=Both [?]=Scenario
        [0]=Objective [K]=Key
        0 1 2 3 4 5 6 7
0 [#] [#] [#] [#] [#] [#] [#] [#]
1 [#] [.] [.] [.] [#] [K] [.] [#]
2 [#] [.] [.] [#] [.] [?] [#] [#]
3 [#] [#] [.] [?] [.] [#] [.] [#]
4 [#] [.] [?] [B] [#] [.] [.] [#]
5 [#] [K] [#] [?] [.] [.] [#] [#]
6 [#] [.] [#] [.] [#] [.] [o] [#]
7 [#] [#] [#] [#] [#] [#] [#] [#]

```

Player Movement

Player movements are calculated based on user input with 4 different options

Input	Direction
N	North (up)
S	South (down)
E	East (right)
W	West (left)

Tile Type Reference

The AdventureMap is a flexible grid of Tiles, each representing a type of state and defined by our legend() method.

Tile Symbol	Tile Type	Definition
[P1]	Player 1	Tile is occupied by Player 1
[P2]	Player 2	Tile is occupied by Player 2
[B]	Both	Occupied by both players
[?]	Scenario	Triggers a scenario to occur
[K]	Key	An item that players must acquire
[O]	Objective	Goal time that must be achieved after acquiring both keys

[]	Empty	A default tile with no special behavior
[#]	Wall	A tile that cannot be occupied

Scenarios

When a player steps onto a Scenario tile (denoted as [?]), the game triggers a scenario event. Each scenario presents a narrative situation and a set of player choices.

Scenario	Description	Mini Game
Ambush	Bandits approach from all sides. Three options are offered: fight the bandits head-on, attempt to buy them off with gold, or use a smoke bomb to escape	Dungeon Escape
Goddess Statue	Players choose between piety or greed towards the goddess who comes with offerings . Negative damage values in the HP column represent healing. Mages excel at respectful options. Warriors can get away with desecration; Rogues receive divine scepticism.	Dungeon Escape
Hazard	A pressure plate, a stone, and a crossbow all act as different hazards. Each option maps neatly to a class archetype.	Dungeon Escape
Treasure	A gold-reward scenario where every class has a path to a positive outcome, but the size of the reward is determined by how well the chosen method matches the player's class.	Dungeon Escape
Medical Supplies	A resource-management scenario with no wrong answer, only tradeoffs. Players choose between immediate healing, banking the supplies as an item for later use, or forgoing the supplies entirely to maintain pace.	Escort Adventure
Falling Rocks	An environmental hazard that emphasises the NPC escort mechanic. Each option trades between protecting the player and protecting the NPC, or taking a slower, safer approach that yields an item.	Escort Adventure

2.4 Minigame One: Dungeon Escape Adventure

A two-player cooperative map traversal game where each player starts separately on the AdventureMap and must navigate using cardinal directions (N/S/E/W) to locate and collect their assigned key, marked as [K] on the grid. Play will alternate between the two players respectively: P1 moves, then P2 moves, and so on.

Along the way, players may land on Scenario tiles [?], which triggers one of the scenario events. This introduces risks and rewards in the form of gold gains or losses, loot items, or potentially fatal outcomes.

Once both players have secured their keys, they must meet at the shared Objective tile [O] to win the game.

2.5 Minigame Two: Escort Adventure

Escort Adventure is a cooperative two-player game where both players and an NPC move together as a single party across the AdventureMap. The goal is to guide the entire party to the Objective tile [O] before anyone's health reaches zero.

Play alternates strictly between the two players: if P1 moves, so will P2, and so on. Since the party travels as a unit, both players are always at the same position on the map — only the active player changes each turn.

Landing on a [?] tile triggers a scenario event. Each scenario tile holds two scenarios — one assigned to P1 and one to P2 — so the event that fires depends on who the active player is that turn. The active player selects a numbered option to resolve it. A scenario tile must be cleared before the party can continue moving through it.

2.6 End

Once a mini game is complete, the command line will congratulate the user. The CLI also asks if a user wants to play another game. If users inputs “n”, the program will terminate.

3. Guide: Adding a Mini Game to the GMAE

3.1 Overview

Mini-adventures can be added with minimal changes to the existing codebase. The system is driven by a single interface (MiniAdventure) which the CLI knows how to launch. Developers only need to implement that interface, render a map using AdventureMap, create and add in any scenarios, and register the new game in the CLI.

GMAECLI Flow

This is an example flow for when a user creates a new mini game.

GMAECLI

- |— creates Players (name + class)
- |— selectAdventure() *user adds a new menu entry here
- |— getAdventureByChoice() *and a case here
- |— TimeObserver.java
- | calls initialize(), then loops:
 - | processInput(input) (ScenarioPrompt)
 - | currentState()

MiniAdventure (interface)

- |— DungeonEscapeAdventure (existing)
- |— EscortAdventure (existing)
- |— NewAdventure *user's new adventure
- |
- |— AdventureMap (shared map engine)
- |— Scenario subclasses (from main.scenarios)

3.2 Understanding GMAE Features

Before writing a new mini-adventure, a user should understand these key features:

- **MiniAdventure**: the interface the new class must implement (package main).
- **AdventureMap**: the grid engine used to build and render the map.
- **TileType**: the enum of available tile types (EMPTY, WALL, KEY, SCENARIO, OBJECTIVE)
- **Scenario subclasses**: the event classes in main.scenarios that can be placed on SCENARIO tiles
- **Status enum**: the shared state enum (INIT, P1MOVE, P2MOVE, P1RESPONSE, P2RESPONSE, END) used to drive the game loop.
- **ScenarioPrompt / ScenarioOutcome**: the return types used to communicate state back to the CLI.

3.3 MiniAdventure Interface Requirements

Every mini-adventure must implement all five methods defined in MiniAdventure. The CLI calls them in a fixed pattern and expects specific return types.

Method	Returns	Responsibility
getName()	String	Returns the display name of the adventure shown in the CLI menu.
initialize(Player, Player)	void	Stores P1 and P2 references, resets all internal state to defaults, then calls setup(). Called once per game session.
setup()	void	Constructs the AdventureMap, places all tiles (walls, keys, scenarios, objective), and populates the scenario registry. May be called again on replay.
processInput(String)	ScenarioPrompt	The main game loop method. Receives raw player input, advances the state machine, and returns the next ScenarioPrompt for the CLI to display. Returns null only when the game has ended.
currentState()	String	Returns a formatted snapshot of the current game — the rendered map, player stats, keys collected, NPC health, and any relevant status message. Called by the CLI after every processInput().

3.4 Step by Step Implementation Guide

3.4.1 Creating the Class

Create a new file in the main package and declare your class as implementing MiniAdventure. Define your map dimensions and any game-specific state fields up front.

3.4.2 Implement getName()

Return the name that will appear in the CLI adventure selection menu.

3.4.3 Implement setUp()

Build the AdventureMap. This involves four tasks: placing border walls, placing any interior maze walls, setting special tiles (KEY, OBJECTIVE, SCENARIO), and registering Scenario objects against their tile coordinates.

3.4.4 Implement initialize(Player, Player)

Store the player references, reset all state variables to their starting values, call setup() to build the map, and then place both players at their starting positions.

3.4.5 Implement processInput(String)

This is the core of your mini-adventure. processInput() is called once per player action and must route the input to the correct handler based on the current Status, then return a ScenarioPrompt telling the CLI what to display next.

The recommended pattern, taken from both existing games, is a switch on a Status enum:

Switch	Method	Description
1	handleInit()	Set currentStatus to P1MOVE, build an introductory message, and return the first move prompt. This fires once on the very first call to processInput()
2	handleMove(String input, Player player, int playerNum)	Parse the direction input, call map.movePlayer(), then branch on the tile type the player landed on. At minimum, handle: WALL/invalid (re-prompt), SCENARIO (trigger the scenario), OBJECTIVE (check win condition), and EMPTY (advance turn).
3	handleScenario(Tile tile, Player player, int playerNum)	Look up the Scenario registered at the tile's coordinates. If the tile has already been cleared, pass through. Otherwise execute the scenario and shift to the RESPONSE state so the player can enter their choice.
4	handleResponse(String input, Player player, Status nextStatus)	Parse the player's numeric choice, call currentScenario.resolve(choice) to get the outcome, apply it to the player, mark the tile cleared, advance the state machine, and return the next move prompt.

5	applyOutcome (Player player, ScenarioOutcome outcome)	A helper that reads the outcome fields and applies them to the player. Always implement this consistently — positive playerDamage reduces health, positive goldAward increases gold, loot items are added to inventory.
---	---	---

3.4.6 Implement currentState()

Return a multi-line String that the CLI prints after each turn. At minimum this should include the rendered map, the legend, and both players' current HP, gold, and score. Add any game-specific status (keys collected, NPC health, etc.) as needed.

3.4.7 Register the adventure in GMAECLI

Two small edits to GMAECLI.java are required to make the new adventure selectable from the menu.

1. Add a menu entry in chooseAdventure()
2. Add a case in getAdventureByChoice()

3.4.8 Adding a New Scenario Type

If the existing scenarios in main.scenarios do not fit the new game, users can create a new one. All scenario classes extend the abstract Scenario base class and follow the same two-method contract used throughout the system.

1. Create a new class in main.scenarios that extends Scenario.
2. In the constructor, call super(name, description) and register options with addOption().
3. Implement execute(Player player): call setPlayer(player) and return a ScenarioPrompt built from getName(), getDescription(), and getOptions().
4. Implement resolve(int choice): switch on choice and delegate to private resolver methods. Each resolver should check activePlayer.getPlayerClass().getType() and return a ScenarioOutcome. Return null for invalid choices.
5. Register the new scenario on a SCENARIO tile in your adventure's setup() method.

4. Design Retrospective

A retrospective on our design: how and why our updated design evolved from our original design. And a reflection of high points, low points, and major challenges across the project.

4.1 Reused System #1: QuestEvent to Scenarios

The Scenario system used across GMAE's mini-adventures is a direct adaptation of the QuestEvent class from Assignment 3. Both represent an event that a player can encounter, carry a loot pool, and produce some reward upon completion.

4.1.2 Source

Bryant Dang
Assignment 3

4.1.3 Repo Structure

QuestEvent	Scenarios
<pre>src/ ├── QuestEvent.java ├── LootStrategy.java ├── FullLootStrategy.java └── TimeObserver.java</pre>	<pre>src/ ├── main/ │ ├── Scenario.java (abstract) │ ├── ScenarioPrompt.java │ ├── ScenarioOutcome.java │ └── scenarios/ │ ├── AmbushScenario.java │ ├── AmbushScenario.java │ ├── FallingRocksScenario.java │ ├── GoddessStatueScenario.java │ ├── HazardScenario.java │ ├── MedicalSuppliesScenario.java │ └── TreasureScenario.java</pre>

4.1.4 Changes

Removed: Time System

QuestEvent implemented a Time System but it was not prevalent in our GMAE since our games were not timed. A Scenario has no concept of time but rather it is an active/inactive state that is determined purely by whether the tile it occupies has been cleared.

Adapted: Participants to ActivePlayer

QuestEvent tracked a dynamic list of UserCharacter participants who could join or leave a quest over time. Loot was distributed across all of them via LootStrategy.

In GMAE, this was reduced to a single field called activePlayer, created by calling setPlayer(Player) in the execute() method. A Scenario only ever acts on one player at a time and the multi-participant system was unnecessary since our GMAE only requires two players.

Adapted: LootStrategy to ScenarioOutcome

QuestEvent used a Strategy pattern for loot distribution: a LootStrategy interface with an injectable implementation (defaulting to FullLootStrategy). Calling rewardCharacters() triggered the strategy to distribute the loot list across all participants.

In our GMAE, the Strategy pattern was replaced with a simpler return-value system where each resolver method in a Scenario subclass constructs and returns a ScenarioOutcome. The adventure class (DungeonEscapeAdventure or EscortAdventure) is responsible for calling applyOutcome() to act on that bundle.

Adapted: Player Choice

QuestEvent had no concept of player choice. A quest simply happened and loot was awarded. The GMAE Scenario introduced executePlayer() and resolve() as the player interaction system.

Maintained: Loot

The one element that we maintained was the loot field: an ArrayList<Item> declared on the event, available to all subclasses. The field name, type, and intent are identical and the only change was how it gets from the event to the player.

4.1.5 Dependencies

Scenarios are used by both mini games as they are a key feature of our AdventureMap and allows the storyline to progress.

DungeonEscapeAdventure: maintains a HashMap<Point, Scenario> with one Scenario per tile coordinate. When a player steps on a SCENARIO tile, the mapped Scenario is executed by the active player and resolved with their numeric choice.

EscortAdventure: maintains two separate HashMaps (p1Scenarios, p2Scenarios) so each tile holds a distinct Scenario per player. The scenario that fires depends on which player is the active mover during that turn. Cleared state is tracked per-player via two HashSets (clearedP1, clearedP2).

4.1.6 Evidence

QuestEvent	Scenarios
<pre>public class QuestEvent implements TimeObserver{ public enum QuestStatus { UPCOMING, ACTIVE, EXPIRED } private String name; private WorldTime worldStartTime; private WorldTime worldEndTime = null; private Realm questRealm; private ArrayList<Item> loot; private ArrayList<UserCharacter> participants; private LootStrategy lootStrategy = new FullLootStrategy(); private QuestStatus status = QuestStatus.UPCOMING; QuestEvent(String name, WorldTime worldStartTime, Realm questRealm) { this.name = name; this.worldStartTime = worldStartTime; this.questRealm = questRealm; this.loot = new ArrayList<Item>(); this.participants = new ArrayList<UserCharacter>(); } }</pre>	<pre>public abstract class Scenario { protected String name; protected String description; protected ArrayList<Item> loot; protected Player activePlayer; protected ArrayList<String> options; public Scenario(String name, String description) { this.name = name; this.description = description; this.loot = new ArrayList<Item>(); this.activePlayer = null; this.options = new ArrayList<String>(); } }</pre>

4.2 Reused System #2: Users to Player Profiles

The GMAE Player model is a direct simplification of the User and Character classes from the prior Python implementation for Assignment 3. In the original system, a User was one account that owned multiple Campaigns and managed many Characters, each with their own inventory. In GMAE, the account and roster concepts were replaced. A Player is a single session entity that combines both User and Character into one class, with game-state fields for the mini-adventures.

4.2.1 Source

4.2.2 Repo Structure

User	Player Profiles
main.py	src/ └─ main/ ├─ PlayerClass.java └─ Player.java

4.2.3 Changes

Adapted: Class and Type Handling

In the previous project, a Character's class was stored as a string with no type safety or branching support. In the GMAE, we created a PlayerClass wrapper holding a Type enum. Scenario outcomes branch on the player's class so the enum makes that branching type-safe.

Adapted: Game States

Three fields were added to Player: health, gold, and score. These exist so that the progression of our storyline can be sufficient due to multiple scenarios being implemented. Compared to the previous User, these fields were not as prevalent to the QuestEvents.

Adapted: Class and Type Handling

In the previous project, a Character's class was stored as a string with no type safety or branching support. In the GMAE, we created a PlayerClass wrapper holding a Type enum. Scenario outcomes branch on the player's class so the enum makes that branching type-safe.

Maintained: Inventory and Item

The Inventory class is identical in both projects where a list of Items has add and remove methods. The only notable difference is that items can be obtained and constructed in new ways. Items are constructed directly with a name, description, and rarity string.

4.2.4 Dependencies

GMAECLI is where Players are created with the name and class from input. It will construct both Player objects and maintain them for the entire session. It also calls giveStartingBonus() directly on each Player before Dungeon Escape begins, adding gold or items to their inventory.

MiniAdventure takes both Players through initialize(Player P1, Player P2) which makes them the core input every game mode requires to start.

DungeonEscapeAdventure stores P1 and P2 as fields and reads from them to render their name, HP, gold, and score in currentState(). It also passes the active player into each scenario via execute(). It writes back to Players through applyOutcome(), which calls other smaller methods depending on the scenario decisions.

EscortAdventure does the same, but additionally checks the active player's inventory for passive items like Bandage, Medkit, SmokeBomb, and ShieldToken, consuming them automatically when triggered. It also calls getMaxHealth() to enforce the HP ceiling during healing.

All six Scenario subclasses receive the active Player via execute() and store them as activePlayer. Each resolver method then calls getPlayerClass().getType() to branch on class and getGold() to check gold thresholds before returning an outcome.

4.2.5 Evidence

Users/Characters	Player
<pre>class Character: def __init__(self, character_id: str, name: str, char_class: str, level: int, user_id: str): self.character_id = character_id self.name = name self.char_class = char_class self.level = level self.user_id = user_id #ADDITIONAL FIELD self.inventory = Inventory() def __str__(self): return f"Character ID: {self.character_id}, Name: {self.name}, Class: {self.char_class}, Level: {self.level}"</pre>	<pre>public Player(String name, PlayerClass playerClass) { this.name = name; this.playerClass = playerClass; this.inventory = new Inventory(); this.gold = 0; this.score = 0; this.health = 100; this.maxHealth = 100; }</pre>
<pre>class User: def __init__(self, user_id: str, username: str): self.user_id = user_id self.username = username self.characters: List[Character] = []</pre>	<pre>public class Player { private String name; private Inventory inventory; private PlayerClass playerClass; private int gold; private int score;</pre>


```
self.campaigns: List[Campaign] = []
```

```
def add_campaign(self, campaign: Campaign):  
    self.campaigns.append(campaign)
```

```
def remove_campaign(self, campaign_id: int):  
    for campaign in self.campaigns:  
        if campaign.campaign_id == campaign_id:  
            self.campaigns.remove(campaign)
```

```
private int health;  
private int maxHealth;  
...
```

4.3 High Points

4.3.1 Defining a Universal System

The biggest architectural high point in GMAE was the MiniAdventure interface which gives every game mode a shared interface. By adapting our previous QuestEvent logic, we established a reusable event system that both Dungeon Escape and Escort Adventure plug into without any major changes. The Scenario library, ScenarioOutcome, and ScenarioPrompt all exist at the system level which was a major plus.

4.3.2 Defining the AdventureMap

The AdventureMap was a really great foundational layer that was used for the entire GMAE implementation. Since it didn't rely on specific game instructions, both adventures were free to build completely different games on top of the same grid engine..

4.3.3 Reusable MiniAdventure Interface

Escort Adventure was influenced by the first EscapeDungeon Adventure and relied on our MiniAdventure Interface. It inherited the full Scenario library, AdventureMap, the Status state machine pattern, and the ScenarioPrompt/ScenarioOutcome pipeline directly from what Dungeon Escape established. This was considered a high point because it allowed the implementation of the second mini game to be a lot easier due to a strong foundation.

4.3.4 Reusable Code from Assignment 3

Player and Inventory are consumed identically across both adventures. Neither game needed a modified or extended player model since they both read and write the same health, gold, score, and inventory fields through the same public methods. The PlayerClass enum in particular is a high point, since both games and all six scenario subclasses branch on it without any casting or string comparisons.

4.4 Low Points

4.4.1 Mini Game Transitions

As we started development for our second game (Escort Adventure), we realized that we needed to fully flush out our foundational system. We had to fully understand how the MiniAdventure interface, AdventureMap grid, and Scenario system worked in order to start brainstorming another game. Escort Adventure handles some of the same problems that Escape Dungeon encountered and we struggled to pinpoint what those issues were.

4.4.2 Defining the AdventureMap

The system is built on three layers with the foundation being the AdventureMap and the MiniAdventure interface sits on top of that. We tried to determine the best possible component that can be reused throughout the program and we decided to create a grid engine that serves as our map for both games. We consider this a big low point for us since we struggled with deciding if this should be hardcoded as a map entity or if it should be dynamic and randomized. We wanted to allocate more time into developing our scenarios so we determined that hardcoding the map (the walls, scenarios, etc.) would be the best option.

4.5 Major Challenges

4.4.1 Code Documentation

We should have followed practical protocol with code documentation since it was a big struggle to have everyone understand/remember the foundational layers. It worked out in the end because we did code on call and we were able to communicate/reflect as we were coding. But in the future and for good practice, making comments as we code would assist our peers immensely.

4.4.2 Technical Issues

We did spend a great deal of time working through our merges. We tried our best to avoid any merge conflicts and had to determine new ways to push our code to the repo. We decided to just force merge a working branch after discussing and testing our code.

5. Gen AI Deliverables

Prompt

Prompt: please provide some software architectural brainstorms that differ slightly from our system. We want to compare and contrast any missing points for enhancement

Response:

1. Formalizing MiniAdventure with an Abstract Class Currently `MiniAdventure` is an interface, which means every game reimplements conventions like the `Status` state machine, turn advancement, and outcome application independently. An abstract class sitting between the interface and the concrete games could codify those shared patterns — providing a default `handleMove()`, a default `applyOutcome()`, and a required `setup()` — while still letting each game override what it needs. `Dungeon Escape` and `Escort Adventure` would become thinner, and adding a third game would be significantly less work. The tradeoff is reduced flexibility; games that need radically different loop structures might fight the base class.

2. Event-Driven Tile System Right now, tile reactions are handled inside each game's `handleMove()` with a switch on `TileType`. An alternative would be to attach a `TileEffect` interface directly to the `Tile` object — so a tile knows what it does when a player lands on it, rather than the game knowing what to do with every tile type. Landing on a `ScenarioTile` would fire itself; landing on a `KeyTile` would auto-collect. This would make `AdventureMap` more self-contained and reduce the per-game branching logic. The downside is that tiles would need references to player state, which creates a coupling that the current design deliberately avoids.

3. Persistent Player Profiles Across Sessions Currently Players are created fresh every session and all state is lost when the program exits — by design. A light persistence layer, even just serializing Player state to a local JSON or text file, would allow scores and gold to carry across runs. This would give returning players a sense of progression, enable leaderboards, and make the starting bonus feel more meaningful since prior performance could influence it. The risk is added complexity in `GMAECLI` and the need to handle corrupted or missing save files gracefully.

4. Separating the Scenario Registry from the Adventure Class Both games embed their scenario registries directly as private fields — a `HashMap<Point, Scenario>` inside `Dungeon Escape`, two `HashMaps` inside `Escort Adventure`. A dedicated `ScenarioRegistry` class that maps coordinates to scenarios, handles cleared state, and supports per-player tracking would centralize all of that logic in one place. The adventure classes would ask the registry what to fire rather than managing it themselves. This would make `Escort Adventure`'s per-player cleared logic reusable for any future game that needs the same mechanic, and would make it much easier to define a map's scenario layout declaratively — potentially even loading it from a config file rather than hardcoding coordinates in `setup()`.

Comparison against the current system:

Enhancement	What it solves	What it adds
Abstract base class	Inconsistent conventions between games	Reduced flexibility per game
Event-driven tiles	Switch logic scattered across games	Tile-to-player coupling
Persistent profiles	No progression between sessions	Save/load complexity

ScenarioRegistry

Registry logic duplicated per game

Another layer to maintain

The two most impactful changes for your specific codebase would be the abstract base class — which directly addresses the low point around transitioning between games — and the [ScenarioRegistry](#) — which addresses the inconsistency in how cleared state is currently tracked between Dungeon Escape and Escort Adventure.

Prompt

Prompt: Edit the README to explain how to run the game

Response: [# INF122](#)

Run the game (Windows / PowerShell)

1. (Optional) Compile the code into `out/`:

```
```powershell
```

```
$files = Get-ChildItem -Recurse -Filter *.java .\src\main
```

```
mkdir out -Force | Out-Null
```

```
javac -d out $files.FullName
```

```
```
```

2. Create the jar manifest (this must be saved to a file):

```
```powershell
```

```
Set-Content -Path manifest.txt -Value "Main-Class: main.GMAECLI" -Encoding ascii
```

```
```
```

3. Build the jar:

```
```powershell
```

```
jar cfm GuildQuest.jar manifest.txt -C out main
```

```
```
```

4. Run it:

```
```powershell
```

```
java -jar GuildQuest.jar
```

```
```
```

Gameplay controls

Input player profiles and select a mini-adventure when prompted.

Player movement uses cardinal directions:

`N` = North, `S` = South, `E` = East, `W` = West.